
Concurrencia y Paralelismo

Bloque I: Concurrencia

Julio 2025

1. Cola de espera [2 puntos]

Vamos a simular una cola de donde cada thread representa una persona que ocupa una posición. Cuando a una persona le toca el turno recibe el aviso de la anterior, hace la acción por la que esperaba turno, y avisa al siguiente. Cada thread solo debe despertar a uno, y no se puede usar una estructura de datos de tipo cola o lista.

Para avisar al siguiente cada thread puede guardar una condición en la estructura compartida que es usada por el siguiente thread para esperar por el aviso. Antes de esperar ese thread sustituye la condición por la suya propia para a su vez poder avisar al thread que venga después de él.

Si no hay ningún thread activo la condición debería estar a NULL.

```
struct wait_row {
    int waiting;          // Number of threads in the queue
    pthread_cond_t *c;    // should be NULL if there is no one waiting or doing the action
    pthread_mutex_t *m;
}

void person(struct wait_row *wr) {
    ...
    act();
    ...
}
```

```
int pthread_mutex_init(pthread_mutex_t *m, pthread_mutex_attr_t *attr);
int pthread_mutex_lock(pthread_mutex_t *m);
int pthread_mutex_trylock(pthread_mutex_t *m);
int pthread_mutex_unlock(pthread_mutex_t *m);
int pthread_cond_init(pthread_cond_t *c, pthread_cond_attr_t *attr);
int pthread_cond_wait(pthread_cond_t *cond, pthread_mutex_t *mtx);
int pthread_cond_signal(pthread_cond_t *cond);
int pthread_cond_broadcast(pthread_cond_t *cond);
pthread_t pthread_self();
```

Solución:

```
struct wait_row {
    int waiting;          // Number of threads in the queue
    pthread_cond_t *c;    // should be NULL if there is no one waiting or doing the action
}

void person(struct wait_row *wr) {
    pthread_cond_t next;
    pthread_cond_t *cur;
    pthread_cond_init(&next, NULL);
    pthread_mutex_lock(wr->m);
    if(wr->c != NULL) {
        waiting++;
        cur = wr->c;
        wr->c = next;
        pthread_cond_wait(cur, wr->m);
        waiting--;
    }
    pthread_mutex_unlock(wr->m);
    act();
    pthread_mutex_lock(wr->m);
    if(wr->waiting > 0) pthread_cond_signal(wr->next);
    else wr->c = NULL;
    pthread_mutex_unlock(wr->m);
}
```

2. Barrera de emparejamiento [1.5 puntos]

Implemente, utilizando los mutex de la librería pthread, una barrera que empareje a los threads de dos tipos A y B en orden de llegada. Esto es, el primer thread en llamar a `pair_barrier_a` y el primer thread en llamar a `pair_barrier_b` se emparejan, y así sucesivamente. Si un thread llega a la barrera y no hay threads del otro tipo tiene que esperar. Las funciones `pair_barrier` deberían devolver el id del thread con el que se ha emparejado.

Para la solución puede usar la cola ya implementada en las funciones `q_init`, `q_insert` y `q_remove`

```
typedef {
    ...
} pair_barrier_t;

void q_init(queue q);
void q_insert(queue q, void *);
void *q_remove(queue q); // returns NULL if the queue is empty

void pair_barrier_init(pair_barrier_t *pb) {
    ...
}

pthread_t pair_barrier_a(pair_barrier_t *pb) {
    ...
}

pthread_t pair_barrier_b(pair_barrier_t *pb) {
    ...
}
```

Solución:

```
typedef {
    pthread_mutex_t m;
    queue qa, qb;
} pair_barrier_t;

typedef {
    pthread_t my_id;
    pthread_t other_id;
    pthread_cond_t c;
} thr_wait_info;

void q_init(queue q);
void q_insert(queue q, void *);
void *q_remove(queue q); // returns NULL if the queue is empty

void pair_barrier_init(pair_barrier_t *pb) {
    pthread_mutex_init(&pb->m, NULL);
    q_init(pb->qa);
    q_init(pb->qb);
}

pthread_t pair_barrier_a(pair_barrier_t *pb) {
    thr_wait_info *other_inf;
    pthread_t other_id;

    pthread_mutex_lock(&pb->m);
    if((other_inf = q_remove(pb->qb)) == NULL) {
        thr_wait_info my_inf;
        pthread_cond_init(&my_inf->c, NULL);
        my_inf->my_id = pthread_self();
        q_insert(pb->qa, &my_inf);
        pthread_cond_wait(&my_inf->c, &pb->m);
        other_id = my_inf->other_id;
    } else {
        other_inf->other_id = pthread_self();
        pthread_cond_signal(other_inf->c);
        other_id = other_inf->my_id;
    }
    pthread_mutex_unlock(&pb->m);

    return other_id;
}

pthread_t pair_barrier_b(pair_barrier_t *pb) {
    // Same as pair_barrier_a, changing qa for qb and viceversa
}
```

3. Línea de procesos [1.5 puntos]

El siguiente módulo permite crear una secuencia de procesos donde cada uno conoce el PID del siguiente.

```
-module(line).
-export([start/1, get_pids/3]).

start(N) ->
    spawn(?MODULE, init, [N-1]).

get_pids(First, A, B) ->
    ...

init(0) ->
    loop(none);
init(N) ->
    Next = spawn(?MODULE, init, [N-1]),
    loop(Next).

loop(Next) ->
    receive
    ...
    end,
    loop(Next).
```

Implemente la función `get_pids/3`, que dado el PID del primer proceso, y dos posiciones A y B en la línea, devuelve los PIDS de los procesos entre las posiciones A y B. Por ejemplo, `get_pids(First, 0, 1)` devolvería una lista con los dos primeros procesos. Puede suponer que dados N procesos en la línea, $0 \leq A \leq B < N$. Las funciones `start/1` e `init/1` no pueden modificarse. Solución:

```
-module(line).
-export([start/1, get_pids/3]).

start(N) ->
    spawn(?MODULE, init, [N-1]).

get_pids(First, A, B) ->
    First ! {get_pids_request, A, B, self()},
    receive
        {get_pids_reply, Pids} -> Pids
    end.

init(0) ->
    loop(none);
init(N) ->
    Next = spawn(?MODULE, init, [N-1]),
    loop(Next).

loop(Next) ->
    receive
        {get_pids_request, 0, 0, From} ->
            From ! {get_pids_reply, [self()]};
        {get_pids_request, 0, B, From} ->
            Next ! {get_pids, B - 1, [self()], From};
        {get_pids_request, A, B, From} ->
            Next ! {get_pids_request, A-1, B-1, From};
        {get_pids, 0, Pids, From} ->
            From ! {get_pids_reply, [self() | lists:reverse(Pids)]};
        {get_pids, B, Pids, From} ->
            Next ! {get_pids, B - 1, [self() | Pids], From}
    end,
    loop(Next).
```
